Building Data Kiosk workflows guide

How to integrate with the Selling Partner API to build and manage Data Kiosk workflows.

This workflow guide describes how to effectively use the Selling Partner API for Data Kiosk (Data Kiosk). With Data Kiosk, you can improve the seller experience by generating accurate business insights.

Data Kiosk's GraphQL-based dynamic reporting suite helps you generate custom GraphQL queries to access bulk data from Amazon datasets. By following the recommended steps, you can easily retrieve, analyze, and organize data, helping sellers make good decisions and understand their businesses better.

> **Note**
>
> As a supplement to this guide, the `SellingPartnerAPIDataKioskSampleApplication` on GitHub provides a full solution that demonstrates how to use the APIs in this guide to construct, retrieve, and store data with AWS services.

API Versions

This guide references operations in the following Selling Partner APIs:

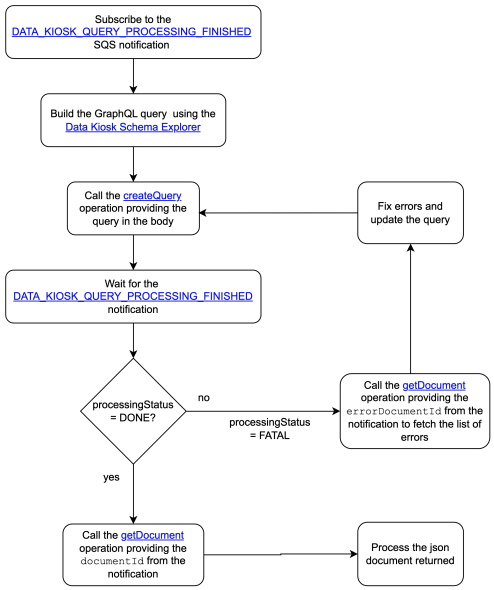
API Reference	API Version	Use Case Guide
Data Kiosk API	11/15/2023	Data Kiosk Use Case Guide

API Reference	API Version	Use Case Guide
Notifications API	v1	Notifications API Use Case Guide

Tutorial: Create and process Data Kiosk queries

> **Note**
>
> Datasets in Data Kiosk are organized collections of data regarding various aspects of the Amazon Selling ecosystem. These Datasets enable users to extract valuable insights to optimize business strategies and operations.
>
> For more information, refer to the Data Kiosk Schema Explorer.

The following diagram highlights the recommended flow for Data Kiosk:



Step 1: Subscribe to Data Kiosk notifications

The `DATA_KIOSK_QUERY_PROCESSING_FINISHED` notification is sent when a Data Kiosk query finishes processing. Both sellers and vendors have the option to subscribe to this notification. The payload of the notification will contain details regarding the returned document, query information, and the associated account information. These notifications are transmitted

Special Authorization Types

INTEGRATION

Building Robust Amazon SP-API Applications

SP-API SDKs

Delete an Application From Your Developer Account

CALLING THE SP-API

Configuration Details

Call Structure

Development Tools

Performance Management

and managed through the Amazon Simple Queue Service (SQS).

To receive and handle `DATA_KIOSK_QUERY_PROCESSING_FINISHED` notifications, it's necessary to subscribe to the queue using the Notifications API. For instructions on configuring a destination and establishing subscriptions, refer to the Notifications API v1 Use Case Guide.

Here is sample code you can use to subscribe to this notification.

Subscribe to a notification
Open Recipe ›

Step 2: Generate the Data Kiosk Query using Schema Explorer

Data Kiosk is a REST API that uses GraphQL query operations for dynamic report functionalities. The Data Kiosk Schema Explorer helps you construct GraphQL queries efficiently. The explorer simplifies query formulation, provides attribute definitions upon hovering, and allows you to select relevant attributes according to your requirements.

For a comprehensive walkthrough on generating queries, refer to the Data Kiosk Schema Explorer User Guide and our Data Kiosk YouTube video.

After you've created your personalized query and chosen the attributes you need, be sure to minify the query and copy it for the next step.

Step 3: Create the Data Kiosk Query for processing

After you are satisfied with your query, call the `createQuery` operation of the Data Kiosk API, passing in the query in the body of the request as a string. Make sure to handle any quotation mark inconsistencies for query validity, which can be done by escaping any nested quotation marks.

After sending the request, the `queryId` will be returned in the response if your request had no errors.

Here is sample code you can use to create the query:

Create Data Kiosk query
Open Recipe ›

> **Note**
>
> The retention of a query varies based on the fields requested. Each field within a schema is annotated with a `@resultRetention` directive that defines how long a query containing that field will be retained. When a query contains multiple fields with different retentions, the shortest (minimum) retention is applied. The retention of a query's resulting documents always matches the retention of the query.

Step 4: Verify that query processing is complete

After calling `createQuery`, Amazon begins processing the query. After processing is complete, the `DATA_KIOSK_QUERY_PROCESSING_FINISHED` notification message is sent to the SQS queue that you subscribed to earlier.

The response can include one of the following:

- A `dataDocumentId` value if data is available as a result of the query.
- An `errorDocumentId` value if there was an error during query processing.
- Neither of these, if no data is returned as a result of the query processing.

For more details on the content of the Data Kiosk notification and an example notification, refer to the Data Kiosk Query Processing Finished Notification Guide.

> **Note**
>
> You can periodically check the processing status using the `getQuery` operation until it's marked as complete (`CANCELLED`, `DONE`, or `FATAL`). If it's still in progress (`IN_PROGRESS` or `IN_QUEUE`), you can keep checking until it's done.

Step 5: Get the processed document details

Edit Your Appstore Listing
Check Listing Status
Appstore Ratings and Reviews
Press Releases and Promotions

SERVICE PROVIDER NETWORK

List Your Service

SECURITY AND COMPLIANCE

SP-API Security and Compliance Overview
Amazon Selling Partner API Guard Implementation Guide
VAT Calculation Service
Amazon Seller Data Access
Best Practices

To access the content of the query result document, use the `getDocument` operation. Provide the `dataDocumentId` or the `errorDocumentId` from the notification as a parameter. This operation will give you a URL that expires in five minutes, allowing access to the document content. If the document is compressed, the `Content-Encoding` header will specify the compression method. Note that this differs from how the Reports API handles compression. Even if the notification returned an `errorDocumentId`, you can still use it with the `getDocument` operation to get a URL for a document containing processing errors.

Here is a code sample you can use to get the processed document details:

Retrieve document details
Open Recipe ›

Step 6: Retrieve document content

To obtain the query document, use the information provided in the previous step. If the document is an error document, address the issues mentioned in the error message, then recreate the query with the corrections.

> **Note**
>
> It's imperative to maintain encryption at rest. Under no circumstances should unencrypted query result document content be stored on disk, even temporarily, as it might contain sensitive information.

Tutorial: Cancel a query in progress

The `cancelQuery` operation is used to cancel a query identified by the `queryId` parameter. It is used when a query is in progress or queued (when `processingStatus` is `IN_QUEUE` or `IN_PROGRESS`). Attempting to cancel a query that has already been terminated (when `processingStatus` is `CANCELLED`) will result in no operation being performed.

When a query is successfully canceled, it will be reflected in subsequent calls to the `getQuery` and `getQueries` operations. This ensures that

A+ CONTENT API

A+ Content API

A+ Content Examples

AMAZON WAREHOUSING AND DISTRIBUTION API

Amazon Warehousing and Distribution API

Amazon Warehousing and Distribution API v2024-05-09 Reference

APP INTEGRATIONS API

App Integrations API

APPLICATION MANAGEMENT API

Application Management API

CATALOG ITEMS API

Catalog Items API

Catalog Items API Rate Limits

Catalog Items API v2022-04-01 Reference

CUSTOMER FEEDBACK API

Customer Feedback API

Customer Feedback API Rate Limits

the status of canceled queries can be retrieved for monitoring and management purposes.

Here is a code sample you can use to cancel a Data Kiosk document in progress:

Cancel active query
Open Recipe ›

Handling processing errors

There are two types of errors in the Data Kiosk API: synchronous errors and asynchronous errors.

- **Synchronous errors:** These errors occur during the initial query creation process using the `createQuery` operation. They prevent the acceptance and further processing of the submission. Typically, synchronous errors are related to syntax issues in the query or mishandled query parameters in the request.
- **Asynchronous errors:** These errors are generated after the submission has been made and initial validation has passed. They occur during processing and are not immediately returned. Asynchronous errors are fetched in the form of error documents. These errors could arise from issues with the content of the query or the data requested. The error message returned provides insights to resolve the problem.

Common errors

400 errors: One common error encountered when using the `createQuery` operation is related to query syntax and validation. If the submitted query contains invalid syntax or includes fields that are not recognized by the API, it can result in a 400 error. To address this issue, you should carefully review the structure and parameters of the query to ensure they comply with the API requirements. Making necessary adjustments to correct any syntax errors before resubmitting the query can help resolve this issue.

429 errors: Another potential error has to do with query concurrency limits. When attempting to create a new query with the `createQuery` endpoint, a 429 error can occur if there's already a query from the same domain in progress. This indicates the API has reached its concurrency limit for handling simultaneous queries from the same domain. To address this, implement

DATA KIOSK

Data Kiosk API

Data Kiosk Schema Explorer User Guide

Data Kiosk Query Processing Finished Notification

Building Data Kiosk workflows guide

Vendor Analytics Dataset Use Case Guide

Schedule Data Kiosk Queries

DELIVERY BY AMAZON API

Delivery by Amazon API

EASY SHIP API

Easy Ship API

EXTERNAL FULFILLMENT

External Fulfillment Inventory API

External Fulfillment Returns API

External Fulfillment Shipping API

External Fulfillment Errors

External Fulfillment APIs Rate Limits

FULFILLMENT BY AMAZON (FBA)

FBA Inventory API

FBA Inventory Dynamic Sandbox Guide

FEEDS API

appropriate handling mechanisms in your application. This could involve either waiting for the in-progress query to complete or canceling it using the `cancelQuery` endpoint before submitting another.

Refer to the Data Kiosk Best Practices section for more help on how to avoid these errors.

Error document analysis and resubmission

When an `errorDocumentId` is returned, it's crucial to retrieve and analyze the error document. This can involve identifying the nature of the error, determining potential fixes, and resubmitting the request with corrected parameters or data.

To access the content of the error document, the process is similar to the standard step of obtaining the processed document, but with the inclusion of the `errorDocumentId` as a parameter instead of `dataDocumentId`. This ensures consistent steps for retrieving and fetching content, but with the specific `errorDocumentId` parameter.

No data availability

An empty `documentId` indicates that no data is available in the report. In such cases, try adjusting the date-time window of the requested data.

Retry policies

Implementing retry policies can be beneficial for handling transient errors, such as temporary network issues. However, it's crucial to apply retries thoughtfully to prevent overwhelming the server with repeated failed requests. Employing exponential backoff is a recommended strategy where the interval between retries increases exponentially with each attempt, mitigating the risk of overloading the server.

Data Kiosk best practices

Query efficiency and optimization

Data Kiosk relies on GraphQL for building queries and retrieving data. GraphQL's ability to request only the needed data minimizes network traffic, enhancing performance. To get the most out of GraphQL, keep these principles in mind:

Feeds API
Feed Type Values
Feeds JSON Schemas
Feeds API Best Practices
Feeds API FAQ
Feeds API Rate Limits
FINANCES
Finances API
Transfers API
FULFILLMENT INBOUND API
Fulfillment Inbound API
Fulfillment Inbound API FAQ
Migrating Fulfillment Inbound workflows
Fulfillment Inbound API Rate Limits
FULFILLMENT OUTBOUND API
Fulfillment Outbound Dynamic Sandbox Guide
Fulfillment Outbound API

- **Optimize data retrieval:** By requesting only necessary fields, you minimize unnecessary data transfer, leading to faster response times and reduced network traffic. Avoid fetching nested structures or additional fields that aren't vital for your application's functionality, as this can bloat response payloads and worsen performance.
- **Filter:** Use GraphQL's filtering capabilities to narrow down query results based on specific criteria. By using filter arguments, you can ensure that only relevant data is returned, reducing the size of response payloads and improving query performance. This targeted approach enhances the efficiency of your application's data retrieval process.
- **Order:** When structuring GraphQL queries, keep in mind that the order of attributes determines the order of returned data in the JSON response. By organizing query attributes in the desired order, you can ensure that data is returned consistently and predictably. This is particularly important when displaying ordered lists or collections in your application.

Handling concurrent requests

Similar to the Reports API, Data Kiosk APIs also have specific request rate limits in place. Exceeding these limits will result in a throttling error, preventing your request from being processed. Data Kiosk rate limits are the same as the Reports API.

Additionally, Data Kiosk processes queries on a first-come, first-served basis. A newly submitted query will only be processed after the previous query has completed its processing. To manage the submission of requests that rely on the completion of preceding ones, consider the following:

- **Implement a request queue:** To manage the sequential processing of requests effectively, maintain a request queue within your application. Whenever a new request is made, add it to the queue. As requests complete processing, check the queue for pending requests and process them in the order they were received. This ensures that requests are handled sequentially and helps prevent race conditions or concurrency issues.

- **Implement cancellation mechanisms:** Allow users to cancel pending requests if they are no longer needed. This improves the responsiveness and usability of your application, as users have more control over the request processing flow. For guidance on implementing this feature, refer to the cancel query tutorial, which includes explanations and sample code.

Updated 19 days ago

← Data Kiosk Query Processing Finished Notification

Vendor Analytics Dataset Use Case Guide →

Listings Items API Rate Limits

Listings Restrictions API

Listings Restrictions API Rate Limits

Manage Product Listings with the Selling Partner API

Building Listings Management Workflows Guide

Understanding Amazon listing status and seller-fulfilled inventory management

Listings Management Workflow Migration

Manage Amazon Haul, Advanced Multiple-Offer, and Multiple-Fulfillment Use Cases

Listings APIs FAQ

Product Type Definitions API

Product Type Definitions API Rate Limits

MERCHANT FULFILLMENT API

Merchant Fulfillment API

MESSAGING API

Messaging API

NOTIFICATIONS API

Notifications API

Notification Type Values

Tutorial: Subscribe to the ORDER_CHANGE Notification

ORDERS API

Orders API

Tutorial: Retrieve and Pass a Purchase Order Number to a Carrier

Orders API Migration Guide

Orders API Rate Limits

PRODUCT FEES API

Product Fees API

PRODUCT PRICING API

Product Pricing API

Product Pricing API and Notifications FAQ

Price Adjustment Automation Workflows Guide

Manage automated pricing rules with SP-API

REPLENISHMENT API

Replenishment API

REPORTS API

Reports API

Report Type Values

Reports JSON Schemas

Reports API FAQ

SALES API

Sales API

SELLER WALLET API

Seller Wallet API

Seller Wallet API Rate Limits

SELLERS API

Sellers API

SERVICES API

Services API

SHIPMENT INVOICING API

Shipment Invoicing API

SHIPPING API

Shipping API v2 Resources

Shipping API v1 Reference

SOLICITATIONS API

Solicitations API

SUPPLY SOURCES API

Supply Sources API

Multi-Location Inventory Integration Guide

Supply Sources API Rate Limits

TOKENS API

Tokens API

UPLOADS API

Uploads API

VEHICLES API

Vehicles API

Vehicles API Rate Limits

VENDOR DIRECT FULFILLMENT APIS

Vendor Direct Fulfillment Dynamic Sandbox Guide

Vendor Direct Fulfillment Workflow Guide

SP-API Bill-to-Party Addresses

VENDOR DIRECT FULFILLMENT TRANSACTION STATUS API

Vendor Direct Fulfillment Transaction Status API

VENDOR DIRECT FULFILLMENT INVENTORY API

Vendor Direct Fulfillment Inventory API

VENDOR DIRECT FULFILLMENT ORDERS API

Vendor Direct Fulfillment Orders API

VENDOR DIRECT FULFILLMENT SHIPPING API

Vendor Direct Fulfillment Shipping API

VENDOR DIRECT FULFILLMENT PAYMENTS API

Vendor Direct Fulfillment Payments API >

VENDOR RETAIL PROCUREMENT INVOICES API

Vendor Retail Procurement Invoices API >

VENDOR RETAIL PROCUREMENT ORDERS API

Vendor Retail Procurement Orders API >

VENDOR RETAIL PROCUREMENT SHIPMENTS API

Vendor Retail Procurement Shipments API >

VENDOR RETAIL PROCUREMENT TRANSACTION STATUS API

Vendor Retail Procurement Transaction Status API >